

CS5275 – The Algorithm Designer's Toolkit  
(S2 AY2025/26)

Lecture 11:

Boolean functions – BLR linearity test

# The finite field of order 2

In this lecture, we focus on Boolean functions

$$f : \mathbb{F}_2^n \rightarrow \mathbb{R},$$

where  $\mathbb{F}_2 = \{0,1\}$  is a the finite field of order 2.

# The finite field of order 2

In this lecture, we focus on Boolean functions

$$f : \mathbb{F}_2^n \rightarrow \mathbb{R},$$

where  $\mathbb{F}_2 = \{0,1\}$  is a the finite field of order 2.

Multiplication is modular 2 (**AND**):

- $0 \cdot 0 = 0$
- $0 \cdot 1 = 0$
- $1 \cdot 0 = 0$
- $1 \cdot 1 = 1$

Addition is modular 2 (**XOR / parity**):

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 0$

# The finite field of order 2

In this lecture, we focus on Boolean functions

$$f : \mathbb{F}_2^n \rightarrow \mathbb{R},$$

where  $\mathbb{F}_2 = \{0,1\}$  is the finite field of order 2.

Multiplication is modular 2 (**AND**):

- $0 \cdot 0 = 0$
- $0 \cdot 1 = 0$
- $1 \cdot 0 = 0$
- $1 \cdot 1 = 1$

Addition is modular 2 (**XOR / parity**):

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 0$

How do we use the tools developed for functions of the form

$$f : \{-1,1\}^n \rightarrow \mathbb{R} ?$$

**Consider the mapping:**

- Viewing  $0 \in \mathbb{F}_2$  as 1
- Viewing  $1 \in \mathbb{F}_2$  as  $-1$

# Linear functions

A function  $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$  is **linear**.

$f(x) = \sum_{i \in [n]} c_i x_i$  for some coefficients  $c_i \in \mathbb{F}_2$ .

$f(x) = \sum_{i \in S} x_i$  for some  $S \subseteq [n]$ .

# Linear functions $\leftrightarrow$ parity functions

A function  $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$  is **linear**.

$$f(x) = \sum_{i \in [n]} c_i x_i \text{ for some coefficients } c_i \in \mathbb{F}_2.$$

$$f(x) = \sum_{i \in S} x_i \text{ for some } S \subseteq [n].$$

$$f(x) = x^S = \chi_S \text{ for some } S \subseteq [n].$$

**Consider the mapping:**

- Viewing  $0 \in \mathbb{F}_2$  as 1
- Viewing  $1 \in \mathbb{F}_2$  as  $-1$

# Distances

$$\text{dist}(f, g) = \Pr[f \neq g]$$

= fraction of inputs on which  $f$  and  $g$  disagrees.

- How do we measure the distance between two Boolean functions?

If we focus on functions  $\mathbb{F}_2^n \rightarrow \mathbb{F}_2$  and viewing them as vectors in  $\mathbb{F}_2^{2^n}$ , then  $\text{dist}(f, g)$  equals  $2^{-n} \cdot \text{Hamming distance}$ .

# Distances

$$\text{dist}(f, g) = \Pr[f \neq g]$$

= fraction of inputs on which  $f$  and  $g$  disagrees.

- How do we measure the distance between two Boolean functions?

If we focus on functions  $\mathbb{F}_2^n \rightarrow \mathbb{F}_2$  and viewing them as vectors in  $\mathbb{F}_2^{2^n}$ , then  $\text{dist}(f, g)$  equals  $2^{-n} \cdot \text{Hamming distance}$ .

**Observation:** For any Boolean functions  $f$  and  $g : \{-1, 1\}^n \rightarrow \{-1, 1\}$ ,  
 $\langle f, g \rangle = \mathbb{E}[fg] = \Pr[f = g] - \Pr[f \neq g] = 1 - 2\text{dist}(f, g).$

# Distances

$$\text{dist}(f, g) = \Pr[f \neq g]$$

= fraction of inputs on which  $f$  and  $g$  disagrees.

- How do we measure the distance between two Boolean functions?

Intuition: Fourier coefficients capture distances to linear functions.

**Corollary:**  $\hat{f}(S) = \langle f, \chi_S \rangle = 1 - 2\text{dist}(f, \chi_S).$

If we focus on functions  $\mathbb{F}_2^n \rightarrow \mathbb{F}_2$  and viewing them as vectors in  $\mathbb{F}_2^{2^n}$ , then  $\text{dist}(f, g)$  equals  $2^{-n} \cdot \text{Hamming distance}$ .

**Observation:** For any Boolean functions  $f$  and  $g : \{-1, 1\}^n \rightarrow \{-1, 1\}$ ,  
 $\langle f, g \rangle = \mathbb{E}[fg] = \Pr[f = g] - \Pr[f \neq g] = 1 - 2\text{dist}(f, g).$

Intuition: Fourier analysis is often useful when a problem involves distances.

# Distance to $\mathcal{P}$

$$\text{dist}(f, g) = \Pr[f \neq g]$$

$$\text{dist}(f, \mathcal{P}) = \min_{g \in \mathcal{P}} \Pr[f \neq g]$$

- A property is a set of functions  $\mathcal{P}$ .  
For example, the collection of linear functions is a property.
- $f$  is  $\epsilon$ -**close** to  $\mathcal{P}$  if  $\text{dist}(f, \mathcal{P}) \leq \epsilon$ .
- $f$  is  $\epsilon$ -**far** from  $\mathcal{P}$  if  $\text{dist}(f, \mathcal{P}) > \epsilon$ .

# Property testing

$$\text{dist}(f, g) = \Pr[f \neq g]$$

$$\text{dist}(f, \mathcal{P}) = \min_{g \in \mathcal{P}} \Pr[f \neq g]$$

- A property is a set of functions  $\mathcal{P}$ .
- $f$  is  $\epsilon$ -**close** to  $\mathcal{P}$  if  $\text{dist}(f, \mathcal{P}) \leq \epsilon$ .
- $f$  is  $\epsilon$ -**far** from  $\mathcal{P}$  if  $\text{dist}(f, \mathcal{P}) > \epsilon$ .

Property testing for  $\mathcal{P}$ :

- **Input:** a function  $f$ .
- **Output:** accept or reject.

Requirement:

- **Soundness:** If  $f$  is  $\epsilon$ -far from  $\mathcal{P}$ , then the algorithm rejects  $f$ .
- **Completeness:** If  $f \in \mathcal{P}$ , then the algorithm accepts  $f$ .

Due to the  $\epsilon$ -slack, this task can often be accomplished in sublinear time.

# Linearity testing

**Our focus:**  $\mathcal{P}$  is the collection of all linear functions :  $\mathbb{F}_2^n \rightarrow \mathbb{F}_2$ .



Property testing for  $\mathcal{P}$ :

- **Input:** a function  $f$ .
- **Output:** accept or reject.

Requirement:

- **Soundness:** If  $f$  is  $\epsilon$ -far from  $\mathcal{P}$ , then the algorithm rejects  $f$ .
- **Completeness:** If  $f \in \mathcal{P}$ , then the algorithm accepts  $f$ .

Due to the  $\epsilon$ -slack, this task can often be accomplished in sublinear time.

# Linearity testing

**Next:** a simple algorithm that uses **only 3 queries** of  $f$ :

- If  $f$  is  $\epsilon$ -far from  $\mathcal{P}$ , then  $f$  is rejected with probability at least  $\epsilon$ .
- If  $f \in \mathcal{P}$ , then  $f$  is accepted with probability 1.



**Our focus:**  $\mathcal{P}$  is the collection of all linear functions :  $\mathbb{F}_2^n \rightarrow \mathbb{F}_2$ .



Property testing for  $\mathcal{P}$ :

- **Input:** a function  $f$ .
- **Output:** accept or reject.

Requirement:

- **Soundness:** If  $f$  is  $\epsilon$ -far from  $\mathcal{P}$ , then the algorithm rejects  $f$ .
- **Completeness:** If  $f \in \mathcal{P}$ , then the algorithm accepts  $f$ .

Due to the  $\epsilon$ -slack, this task can often be accomplished in sublinear time.

# Linearity testing

Can amplify the success probability to  $1 - \delta$  by repeating for  $O\left(\frac{\log \frac{1}{\delta}}{\epsilon}\right)$  times.

**Next:** a simple algorithm that uses only 3 queries of  $f$ :

- If  $f$  is  $\epsilon$ -far from  $\mathcal{P}$ , then  $f$  is rejected with probability at least  $\epsilon$ .
- If  $f \in \mathcal{P}$ , then  $f$  is accepted with probability 1.

**Our focus:**  $\mathcal{P}$  is the collection of all linear functions :  $\mathbb{F}_2^n \rightarrow \mathbb{F}_2$ .

Property testing for  $\mathcal{P}$ :

- **Input:** a function  $f$ .
- **Output:** accept or reject.

Requirement:

- **Soundness:** If  $f$  is  $\epsilon$ -far from  $\mathcal{P}$ , then the algorithm rejects  $f$ .
- **Completeness:** If  $f \in \mathcal{P}$ , then the algorithm accepts  $f$ .

Due to the  $\epsilon$ -slack, this task can often be accomplished in sublinear time.

# Blum–Luby–Rubinfeld linearity test

**BLR test:**

- Choose  $\mathbf{x} \sim \mathbb{F}_2^n$  and  $\mathbf{y} \sim \mathbb{F}_2^n$  independently.
- Query  $f$  at  $\mathbf{x}$ ,  $\mathbf{y}$ , and  $\mathbf{x} + \mathbf{y}$ .
- Accept if  $f(\mathbf{x}) + f(\mathbf{y}) = f(\mathbf{x} + \mathbf{y})$ .

# Blum–Luby–Rubinfeld linearity test

## BLR test:

- Choose  $\mathbf{x} \sim \mathbb{F}_2^n$  and  $\mathbf{y} \sim \mathbb{F}_2^n$  independently.
- Query  $f$  at  $\mathbf{x}$ ,  $\mathbf{y}$ , and  $\mathbf{x} + \mathbf{y}$ .
- Accept if  $f(\mathbf{x}) + f(\mathbf{y}) = f(\mathbf{x} + \mathbf{y})$ .

**Completeness:** If  $f \in \mathcal{P}$ , then the algorithm accepts  $f$  with probability 1.

If  $f$  is linear, then  $f(x) + f(y) = f(x + y)$  for any  $x$  and  $y$ .



# Blum–Luby–Rubinfeld linearity test

## BLR test:

- Choose  $\mathbf{x} \sim \mathbb{F}_2^n$  and  $\mathbf{y} \sim \mathbb{F}_2^n$  independently.
- Query  $f$  at  $\mathbf{x}$ ,  $\mathbf{y}$ , and  $\mathbf{x} + \mathbf{y}$ .
- Accept if  $f(\mathbf{x}) + f(\mathbf{y}) = f(\mathbf{x} + \mathbf{y})$ .

Addition becomes multiplication

We view  $f$  as a function  $\mathbb{F}_2^n \rightarrow \{-1, 1\}$ :

- Viewing  $0 \in \mathbb{F}_2$  as 1
- Viewing  $1 \in \mathbb{F}_2$  as  $-1$



Thus, we can apply Fourier analysis.

**Completeness:** If  $f \in \mathcal{P}$ , then the algorithm accepts  $f$  with probability 1.

**Soundness:** If  $f$  is  $\epsilon$ -far from  $\mathcal{P}$ , then the algorithm rejects  $f$  with probability at least  $\epsilon$ .

Algorithm accepts  $f$ .

$f(\mathbf{x})f(\mathbf{y}) = f(\mathbf{x} + \mathbf{y})$

$f(\mathbf{x})f(\mathbf{y})f(\mathbf{x} + \mathbf{y}) = 1$

# Blum–Luby–Rubinfeld linearity test

$$\begin{aligned} & \Pr[\text{algorithm accepts } f] \\ &= \mathbb{E}_{x,y} \left[ \frac{1}{2} + \frac{1}{2} f(x)f(y)f(x+y) \right] \\ &= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x \left[ f(x) \mathbb{E}_y [f(y)f(x+y)] \right] \end{aligned}$$

Algorithm accepts  $f$ .

$f(x)f(y) = f(x+y)$

$f(x)f(y)f(x+y) = 1$

# Blum–Luby–Rubinfeld linearity test

$\Pr[\text{algorithm accepts } f]$

$$= \mathbb{E}_{x,y} \left[ \frac{1}{2} + \frac{1}{2} f(x) f(y) f(x+y) \right]$$

$$= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x \left[ f(x) \mathbb{E}_y [f(y) f(x+y)] \right]$$

$$= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x [f(x) \cdot (f * f)(x)] \quad \leftarrow$$

For any two functions  $f, g : \mathbb{F}_2^n \rightarrow \mathbb{R}$ , their **convolution** is  
 $(f * g)(x) = \mathbb{E}_y [f(y) g(x+y)] = \mathbb{E}_y [f(y) g(x-y)]$ .

# Blum–Luby–Rubinfeld linearity test

$\Pr[\text{algorithm accepts } f]$

$$\begin{aligned} &= \mathbb{E}_{x,y} \left[ \frac{1}{2} + \frac{1}{2} f(x) f(y) f(x+y) \right] \\ &= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x \left[ f(x) \mathbb{E}_y [f(y) f(x+y)] \right] \\ &= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x [f(x) \cdot (f * f)(x)] \\ &= \frac{1}{2} + \frac{1}{2} \sum_{S \subseteq [n]} \hat{f}(S) \cdot \widehat{f * f}(S) \quad \leftarrow \\ &= \frac{1}{2} + \frac{1}{2} \sum_{S \subseteq [n]} \hat{f}(S)^3 \quad \leftarrow \end{aligned}$$

For any two functions  $f, g : \mathbb{F}_2^n \rightarrow \mathbb{R}$ , their **convolution** is  $(f * g)(x) = \mathbb{E}_y [f(y) g(x+y)] = \mathbb{E}_y [f(y) g(x-y)]$ .

**Plancherel's theorem:**  $\mathbb{E}[fg] = \sum_{S \subseteq [n]} \hat{f}(S) \hat{g}(S)$

**Lemma:**  $\widehat{f * g}(S) = \hat{f}(S) \hat{g}(S)$

# Blum–Luby–Rubinfeld linearity test

Pr[algorithm accepts  $f$ ]

$$= \mathbb{E}_{x,y} \left[ \frac{1}{2} + \frac{1}{2} f(x) f(y) f(x+y) \right]$$

$$= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x \left[ f(x) \mathbb{E}_y [f(y) f(x+y)] \right]$$

$$= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x [f(x) \cdot (f * f)(x)]$$

$$= \frac{1}{2} + \frac{1}{2} \sum_{S \subseteq [n]} \hat{f}(S) \cdot \widehat{f * f}(S)$$

$$= \frac{1}{2} + \frac{1}{2} \sum_{S \subseteq [n]} \hat{f}(S)^3$$

Since the range of the output of  $f$  is  $\{-1, 1\}$ ,

$$\leq \frac{1}{2} + \frac{1}{2} \max_{S \subseteq [n]} \hat{f}(S) \quad \leftarrow \sum_{S \subseteq [n]} \hat{f}(S)^2 = 1$$

$$\leq \frac{1}{2} + \frac{1}{2} \max_{S \subseteq [n]} (1 - 2 \text{dist}(f, \chi_S)) \quad \leftarrow \hat{f}(S) = 1 - 2 \text{dist}(f, \chi_S)$$

# Blum–Luby–Rubinfeld linearity test

$$\begin{aligned} & \Pr[\text{algorithm accepts } f] \\ &= \mathbb{E}_{x,y} \left[ \frac{1}{2} + \frac{1}{2} f(x)f(y)f(x+y) \right] \\ &= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x \left[ f(x) \mathbb{E}_y [f(y)f(x+y)] \right] \\ &= \frac{1}{2} + \frac{1}{2} \mathbb{E}_x [f(x) \cdot (f * f)(x)] \\ &= \frac{1}{2} + \frac{1}{2} \sum_{S \subseteq [n]} \hat{f}(S) \cdot \widehat{f * f}(S) \\ &= \frac{1}{2} + \frac{1}{2} \sum_{S \subseteq [n]} \hat{f}(S)^3 \\ &\leq \frac{1}{2} + \frac{1}{2} \max_{S \subseteq [n]} \hat{f}(S) \\ &\leq \frac{1}{2} + \frac{1}{2} \max_{S \subseteq [n]} (1 - 2\text{dist}(f, \chi_S)) \\ &< \textcolor{red}{1} - \textcolor{red}{\epsilon} \end{aligned}$$

**Soundness:** If  $f$  is  $\epsilon$ -far from  $\mathcal{P}$ , then the algorithm rejects  $f$  with probability at least  $\epsilon$ .

$f$  is  $\epsilon$ -far from being linear  $\rightarrow \text{dist}(f, \chi_S) > \epsilon$

# Blum–Luby–Rubinfeld linearity test

- **Summary:**

- BLR test decides whether  $f$  is close to some linear function  $\chi_S$ .
- However, the test does not reveal the function  $\chi_S$ .



**Exercise:**  $n$  queries are necessary and sufficient to determine  $\chi_S$ .

# Local correction

- **Summary:**

- BLR test decides whether  $f$  is close to some linear function  $\chi_S$ .
  - However, the test does not reveal the function  $\chi_S$ .
- 

**Exercise:**  $n$  queries are necessary and sufficient to determine  $\chi_S$ .

## Local correctability:

- Suppose  $f : \mathbb{F}_2^n \rightarrow \{-1, 1\}$  is  $\epsilon$ -close to  $\chi_S$  for some unknown  $S$ .
- For any input  $x \in \mathbb{F}_2^n$ :
  - $\chi_S(x)$  can be determined using 3 queries to  $f$  with probability at least  $1 - 2\epsilon$ .

# Local correction

## Local correctability:

- Suppose  $f : \mathbb{F}_2^n \rightarrow \{-1, 1\}$  is  $\epsilon$ -close to  $\chi_S$  for some unknown  $S$ .
- For any input  $x \in \mathbb{F}_2^n$ :
  - $\chi_S(x)$  can be determined using 3 queries to  $f$  with probability at least  $1 - 2\epsilon$ .

- Choose  $\mathbf{y} \sim \mathbb{F}_2^n$ .
- Query  $f$  at  $\mathbf{y}$ , and  $x + \mathbf{y}$ .
- Output  $f(\mathbf{y})f(x + \mathbf{y})$ .

# Local correction

$$\Pr[f(\mathbf{y}) \neq \chi_S(\mathbf{y})] \leq \epsilon$$

$$\Pr[f(x + \mathbf{y}) \neq \chi_S(x + \mathbf{y})] \leq \epsilon$$

$$\Pr[f \neq \chi_S] \leq \epsilon$$

Both  $\mathbf{y}$  and  $x + \mathbf{y}$  are uniformly distributed on  $\mathbb{F}_2^n$ .

## Local correctability:

- Suppose  $f : \mathbb{F}_2^n \rightarrow \{-1, 1\}$  is  $\epsilon$ -close to  $\chi_S$  for some unknown  $S$ .
- For any input  $x \in \mathbb{F}_2^n$ :
  - $\chi_S(x)$  can be determined using 3 queries to  $f$  with probability at least  $1 - 2\epsilon$ .

- Choose  $\mathbf{y} \sim \mathbb{F}_2^n$ .
- Query  $f$  at  $\mathbf{y}$ , and  $x + \mathbf{y}$ .
- Output  $f(\mathbf{y})f(x + \mathbf{y})$ .

# Local correction

$$f(\mathbf{y})f(x + \mathbf{y}) = \chi_S(\mathbf{y})\chi_S(x + \mathbf{y}) = \chi_S(x)$$

conditioning on these bad events not happening

$$\Pr[f(\mathbf{y}) \neq \chi_S(\mathbf{y})] \leq \epsilon$$

$$\Pr[f(x + \mathbf{y}) \neq \chi_S(x + \mathbf{y})] \leq \epsilon$$

$$\Pr[f \neq \chi_S] \leq \epsilon$$

Both  $\mathbf{y}$  and  $x + \mathbf{y}$  are uniformly distributed on  $\mathbb{F}_2^n$ .

## Local correctability:

- Suppose  $f : \mathbb{F}_2^n \rightarrow \{-1, 1\}$  is  $\epsilon$ -close to  $\chi_S$  for some unknown  $S$ .
- For any input  $x \in \mathbb{F}_2^n$ :
  - $\chi_S(x)$  can be determined using 3 queries to  $f$  with probability at least  $1 - 2\epsilon$ .

- Choose  $\mathbf{y} \sim \mathbb{F}_2^n$ .
- Query  $f$  at  $\mathbf{y}$ , and  $x + \mathbf{y}$ .
- Output  $f(\mathbf{y})f(x + \mathbf{y})$ .

# Fourier analysis of convolution

- To finish the proof, it remains to prove the lemma:

$$\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$$

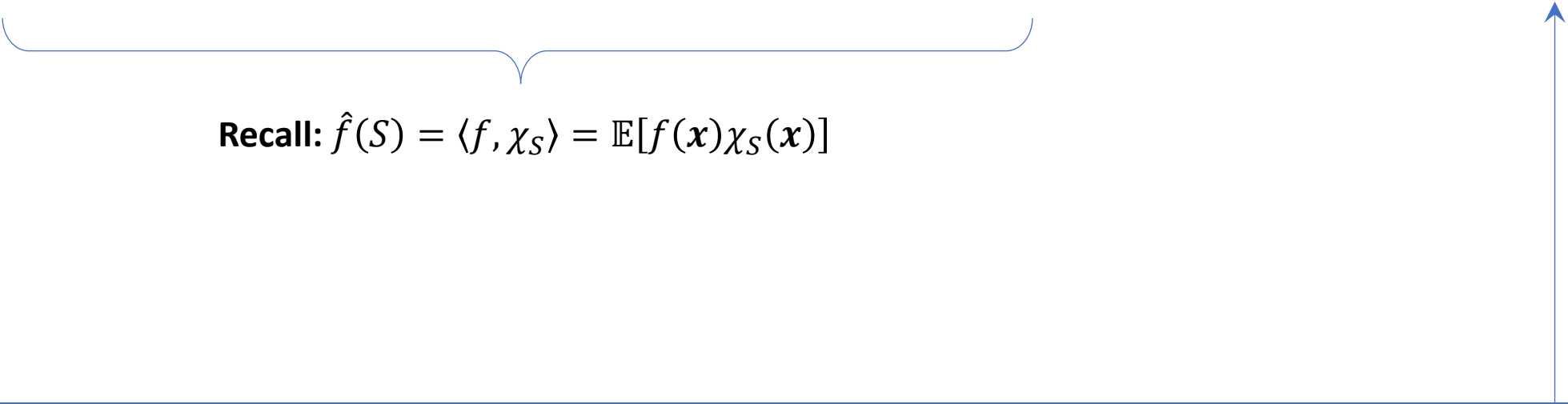
For any two functions  $f, g : \mathbb{F}_2^n \rightarrow \mathbb{R}$ , their **convolution** is  $(f * g)(x) = \mathbb{E}_{\mathbf{y}}[f(\mathbf{y})g(x + \mathbf{y})] = \mathbb{E}_{\mathbf{y}}[f(\mathbf{y})g(x - \mathbf{y})]$ .

# Fourier analysis of convolution

- To finish the proof, it remains to prove the lemma:

$$\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$$

$$\widehat{f * g}(S) = \langle f * g, \chi_S \rangle = \mathbb{E}_{\mathbf{x} \sim \mathbb{F}_2^n}[(f * g)(\mathbf{x}) \cdot \chi_S(\mathbf{x})] = \mathbb{E}_{\mathbf{x} \sim \mathbb{F}_2^n}[\mathbb{E}_{\mathbf{y} \sim \mathbb{F}_2^n}[f(\mathbf{y})g(\mathbf{x} - \mathbf{y})] \cdot \chi_S(\mathbf{x})]$$



**Recall:**  $\hat{f}(S) = \langle f, \chi_S \rangle = \mathbb{E}[f(\mathbf{x})\chi_S(\mathbf{x})]$

For any two functions  $f, g : \mathbb{F}_2^n \rightarrow \mathbb{R}$ , their **convolution** is  $(f * g)(\mathbf{x}) = \mathbb{E}_{\mathbf{y}}[f(\mathbf{y})g(\mathbf{x} + \mathbf{y})] = \mathbb{E}_{\mathbf{y}}[f(\mathbf{y})g(\mathbf{x} - \mathbf{y})]$ .

# Fourier analysis of convolution

- To finish the proof, it remains to prove the lemma:

$$\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$$

$$\widehat{f * g}(S) = \langle f * g, \chi_S \rangle = \mathbb{E}_{\mathbf{x} \sim \mathbb{F}_2^n}[(f * g)(\mathbf{x}) \cdot \chi_S(\mathbf{x})] = \mathbb{E}_{\mathbf{x} \sim \mathbb{F}_2^n}[\mathbb{E}_{\mathbf{y} \sim \mathbb{F}_2^n}[f(\mathbf{y})g(\mathbf{x} - \mathbf{y})] \cdot \chi_S(\mathbf{x})]$$

$$= \mathbb{E}_{\mathbf{y} \sim \mathbb{F}_2^n, \mathbf{z} \sim \mathbb{F}_2^n}[f(\mathbf{y})g(\mathbf{z}) \cdot \chi_S(\mathbf{y} + \mathbf{z})] \quad \mathbf{z} = \mathbf{x} - \mathbf{y}$$

$$= \mathbb{E}_{\mathbf{y} \sim \mathbb{F}_2^n, \mathbf{z} \sim \mathbb{F}_2^n}[f(\mathbf{y})\chi_S(\mathbf{y}) \cdot g(\mathbf{z})\chi_S(\mathbf{z})] \quad \chi_S(\mathbf{y})\chi_S(\mathbf{z}) = \chi_S(\mathbf{y} + \mathbf{z})$$

For any two functions  $f, g : \mathbb{F}_2^n \rightarrow \mathbb{R}$ , their **convolution** is  $(f * g)(x) = \mathbb{E}_{\mathbf{y}}[f(\mathbf{y})g(x + \mathbf{y})] = \mathbb{E}_{\mathbf{y}}[f(\mathbf{y})g(x - \mathbf{y})]$ .

# Fourier analysis of convolution

- To finish the proof, it remains to prove the lemma:

$$\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$$

$$\begin{aligned}\widehat{f * g}(S) &= \langle f * g, \chi_S \rangle = \mathbb{E}_{\mathbf{x} \sim \mathbb{F}_2^n}[(f * g)(\mathbf{x}) \cdot \chi_S(\mathbf{x})] = \mathbb{E}_{\mathbf{x} \sim \mathbb{F}_2^n}[\mathbb{E}_{\mathbf{y} \sim \mathbb{F}_2^n}[f(\mathbf{y})g(\mathbf{x} - \mathbf{y})] \cdot \chi_S(\mathbf{x})] \\&= \mathbb{E}_{\mathbf{y} \sim \mathbb{F}_2^n, \mathbf{z} \sim \mathbb{F}_2^n}[f(\mathbf{y})g(\mathbf{z}) \cdot \chi_S(\mathbf{y} + \mathbf{z})] && \mathbf{z} = \mathbf{x} - \mathbf{y} \\&= \mathbb{E}_{\mathbf{y} \sim \mathbb{F}_2^n, \mathbf{z} \sim \mathbb{F}_2^n}[f(\mathbf{y})\chi_S(\mathbf{y}) \cdot g(\mathbf{z})\chi_S(\mathbf{z})] && \chi_S(\mathbf{y})\chi_S(\mathbf{z}) = \chi_S(\mathbf{y} + \mathbf{z}) \\&= \mathbb{E}_{\mathbf{y} \sim \mathbb{F}_2^n}[f(\mathbf{y})\chi_S(\mathbf{y})] \cdot \mathbb{E}_{\mathbf{z} \sim \mathbb{F}_2^n}[g(\mathbf{z})\chi_S(\mathbf{z})] \\&= \hat{f}(S)\hat{g}(S)\end{aligned}$$

For any two functions  $f, g : \mathbb{F}_2^n \rightarrow \mathbb{R}$ , their **convolution** is  $(f * g)(x) = \mathbb{E}_{\mathbf{y}}[f(\mathbf{y})g(x + \mathbf{y})] = \mathbb{E}_{\mathbf{y}}[f(\mathbf{y})g(x - \mathbf{y})]$ .

# Motivation

- BLR test was originally designed for **program checkers** and **self-correctors**.

BLR test allows us to quickly check whether a program correctly computes a linear function without running it on every input.

BLR test offers a mechanism that can recover the correct output even if the program is slightly faulty.

# Further developments

- The BLR test introduced a **powerful paradigm**:

Global properties can be verified using only a few random local checks.

- Instead of examining an entire object:
  - Randomly sample a small number of positions.
  - Check whether local consistency conditions hold.
  - Conclude whether the object is likely correct or far from correct
- This idea influenced several areas in theoretical computer science.

# Further developments

## **Locally testable code:**

- In coding theory, a codeword represents encoded data.
- A locally testable code allows us to check whether a string is:
  - a valid codeword, or
  - far from any valid codewordby reading only a few positions of the string.

# Further developments

## **Locally testable code:**

- In coding theory, a codeword represents encoded data.
- A locally testable code allows us to check whether a string is:
  - a valid codeword, or
  - far from any valid codewordby reading only a few positions of the string.

## **Probabilistically checkable proof (PCP):**

- A probabilistically checkable proof allows a verifier to check the correctness of a proof by reading only a few randomly chosen bits.

# PCP theorem

$$\text{NP} = \text{PCP}(O(\log n), O(1))$$

A problem is in NP if there exists a polynomial-time verification algorithm such that:

- **Completeness:** For every yes-instance, there exists a certificate that the verifier accepts.
- **Soundness:** For every no-instance, all certificates are rejected by the verifier.

A problem is in  $\text{PCP}(r, s)$  if there exists a polynomial-time verification algorithm that uses  $r$  random bits to query  $s$  bits of the certificate such that:

- **Completeness:** For every yes-instance, there exists a certificate that the verifier accepts with probability 1.
- **Soundness:** For every no-instance, all certificates are rejected by the verifier with probability at least  $1/2$ .

# PCP theorem

$$\text{NP} = \text{PCP}(O(\log n), O(1))$$



Many applications to hardness of approximation

**Exercise:** There exists a constant  $\epsilon > 0$  such that it is **NP-hard** to distinguish between:

- Instances of 3SAT that are satisfiable.
- instances of 3SAT in which at most a  $(1 - \epsilon)$ -fraction of clauses can be satisfied.

# Outlook

- **Next:** Applying Fourier analysis to the analysis of voting rules.



+ impossibility results



also known as social choice functions

# References

- Sections 1.5–1.6 of <https://arxiv.org/abs/2105.10386>